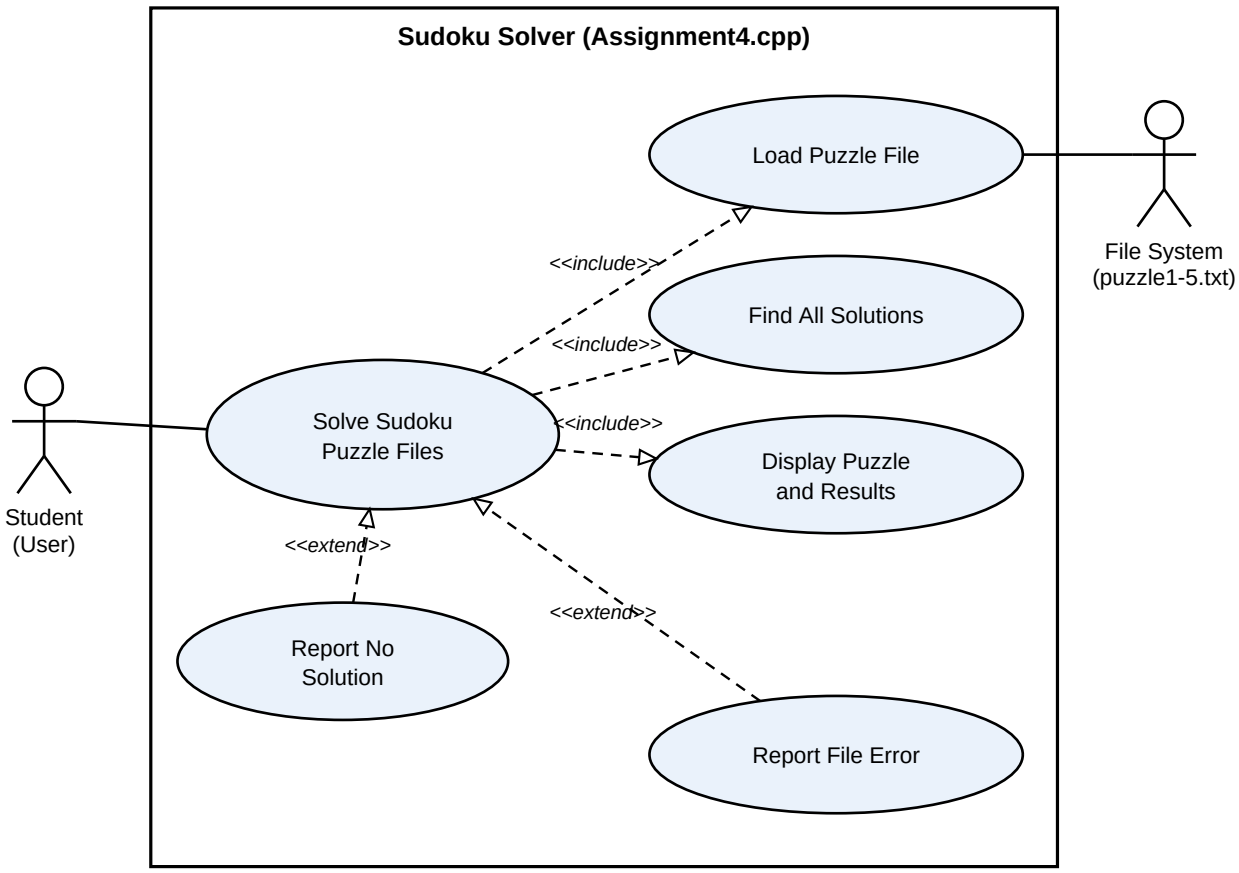
UML Use Case Model

EECS 348 Assignment 4 – Recursive Sudoku Solver

Use Case Diagram



Legend: Solid lines are associations between an actor and a use case. Dashed arrows are «include» (base use case always uses the included one; arrow points to the included use case) and «extend» (optional behavior that triggers under a condition; arrow points to the base use case).

a. Identified Use Case

Solve Sudoku Puzzle Files (ID: UC-1)

Primary actor: Student (the person who runs the program)
Supporting actor: File System (supplies puzzle1.txt through puzzle5.txt)
Included use cases: Load Puzzle File, Find All Solutions, Display Puzzle and Results
Extending use cases: Report No Solution, Report File Error

b. Brief Description

The Student runs the program, which reads each of the five Sudoku puzzle files, uses a recursive depth-first search with backtracking to find every valid solution, and prints the file name, the original puzzle, and all solutions found (or “No solution found”) to the screen.

c. Step-by-Step Flow of Events (Outline)

Main flow (repeated for each of puzzle1.txt – puzzle5.txt):

1. The Student launches the program (Assignment4.exe).
2. The system opens the next puzzle file from the File System.
3. The system reads 81 tokens into a 9x9 grid (a digit is a given clue; an underscore is a blank).
4. The system checks that the starting clues do not already violate a row, column, or block rule.
5. The system prints the puzzle file name and the puzzle as read from the file.
6. The system finds the first empty cell and determines which digits 1–9 are legal there.
7. The system places a candidate digit in the cell and recursively solves the grid with one more cell filled.
8. If the recursion fails, the system removes the digit (backtracks) and tries the next candidate.
9. When no empty cells remain, the system records the completed grid as a solution and keeps searching for others.
10. When every candidate has been tried, the system prints each solution found.
11. The system moves on to the next puzzle file; after the fifth file, the program ends.

Alternate flows:

A1 – File cannot be opened or is incomplete (step 2 or 3): the system reports a file error for that file and continues with the next puzzle file.

A2 – Clues contradict each other (step 4): the system skips the search and reports “No solution found.”

A3 – Search exhausts all candidates with no complete grid (step 10): the system prints “No solution found.”

A4 – More than one solution exists (step 10): the system prints every solution, not just the first.

d. Detailed Description

Use case name	Solve Sudoku Puzzle Files
Use case ID	UC-1
Primary actor	Student (user)

Supporting actor	File System (puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt)
Goal	Obtain every valid solution for each of the five Sudoku puzzles, with clear output.
Preconditions	The program has been compiled on the EECS cycle server. The five puzzle files are in the same directory as the executable. Each file holds 9 rows of 9 space-separated tokens, where a token is a digit 1–9 or an underscore.
Trigger	The Student runs the program.
Postconditions (success)	For every readable puzzle, the screen shows the file name, the original puzzle, and each solution found. Every printed solution has each digit 1–9 exactly once in every row, column, and 3x3 block.
Postconditions (failure)	For an unsolvable puzzle, “No solution found” is printed. For an unreadable file, a file error is reported. In both cases the remaining files are still processed.
Included behavior	**Load Puzzle File:** open the file and fill the 9x9 grid. **Find All Solutions:** recursive depth-first search with backtracking. Pick an empty cell, compute its candidates from its row, column, and block, try each candidate, recurse, and undo it on failure. A full grid is saved as a solution, then the search continues so that non-unique puzzles yield all of their solutions. **Display Puzzle and Results:** print the file name, the starting puzzle, and every solution.
Extending behavior	**Report No Solution:** runs when the search finds zero solutions or the clues conflict. **Report File Error:** runs when a puzzle file is missing or has fewer than 81 tokens.
Business rules	Each digit 1–9 must appear exactly once per row, column, and block. The algorithm must be a recursive brute-force search written with objects, not free functions. All solutions must be printed if there is more than one.
Frequency of use	Once per run of the program; processes five puzzles per run.
Assumptions	Puzzle files are well formed apart from the error cases above. Puzzles are small enough that brute force finishes in well under a second.